

UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
INFORMATICA

Migrazione e sviluppo di un'applicazione Android per l'analisi dei melanoma

Relatore:
Prof. Ignazio Gallo

Tesi di Laurea di:
Gabriele Caretti
Matricola 756564

Anno accademico:
2025/2026

Indice

1 Introduzione **1**
1.1 Obiettivo 1
1.2 Melanoma 1

2 Analisi dei Requisiti **3**

3 Progettazione **5**
3.1 Descrizione generale 5
3.2 Scelte architetturali 5
3.2.1 Migrazione verso Kotlin Multiplatform 5
3.2.2 Architettura dell'applicazione 6
3.3 Specifica dei requisiti 7
3.3.1 Interfaccia utente 7
3.3.2 Realizzazione del layout 7
3.4 Database 7
3.4.1 Modello dati 7

4 Sviluppo **8**
4.1 Creazione base dei dati 8
4.2 Sicurezza 8

5 Conclusioni **9**

Capitolo 1

Introduzione

1.1 Obiettivo

Il progetto di tesi si propone di sviluppare un'applicazione mobile cross-platform, compatibile sia su piattaforme Android che iOS, partendo dallo sviluppo di un'applicazione nativa scritta in Android Kotlin, che necessità di una migrazione verso Kotlin Multiplatform. L'applicazione nativa offre analisi tramite l'input di parametri dei tumori di tipo Melanoma, con metrica Melanoma TNM, seguendo il manuale AJCC 8th Edition. Questa sarà successivamente estesa con l'implementazione di analisi di tipo cSCC Testa-Collo, con metrica cSCC TNM, sempre seguendo il precedente manuale AJCC 8th Edition.

L'obiettivo è quello di offrire uno strumento semplice e veloce per semplificare il lavoro dei medici nella fase di analisi dei tumori, con dati sincronizzati in ambiente cloud tramite app installabili su dispositivi mobili.

Il processo di realizzazione del software è strutturata in una sequenza lineare di fasi, nello specifico:

- Analisi dei requisiti;
- Progettazione;
- Sviluppo;
- Conclusioni;

1.2 Melanoma

Il melanoma è la forma più aggressiva e pericolosa di tumore cutaneo. Si origina dai *melanociti*, cellule presenti sull'epidermide responsabili della produzione di

melanina, ossia il pigmento che ne da il colore e la protegge dai raggi solari, come ultravioletti (UVA, UVB e UVC). Quando il DNA di queste cellule si danneggia (spesso a causa di un'eccessiva esposizione ai componenti pericolosi dei raggi solari), i melanociti possono iniziare a moltiplicarsi in modo incontrollato, formando quindi una massa tumorale.

A livello visivo, un melanoma può manifestarsi come la comparsa di un "neo" anomalo oppure come il cambiamento di forma, colore o dimensione di uno già esisteste. Sebbene i melanomi rappresentino solamente una piccola percentuale di tutti i tumori cutanei, questi sono responsabili della maggioranza dei decessi legati a tal genere di patologie. Questo è spesso dovuto alla loro tendenza di diffondersi nei tessuti profondi e a generare metastasi in altri organi se non individuati in tempo.

Una diagnosi precoce e precisa può quindi salvare la vita di un individuo affetto da melanoma. In ambito clinico, questo viene valutato attraverso il **sistema di stadiazione TNM** (Tumore, LinfoNodi, Metastasi), un metodo standardizzato ed esposto in manuali come l'AJCC 8th Edition menzionato in precedenza, che ne definiscono con importanza critica la prognosi e il piano terapeutico del paziente.

In un panorama ampio come quello dell'oncologia dermatologica, un'altra importante manifestazione di tale patologia è il **Carcinoma a Cellule Squamose (SCC)**. Quest'ultimo, specialmente nei casi in cui è localizzato nel distretto testa-collo, richiede un monitoraggio costante e attento, a causa del suo potenziale impatto distruttivo a livello funzionale ed estetico.

Questo progetto di applicazione mobile, mira quindi allo sviluppo di uno strumento di supporto digitale, che ne semplifica le fasi di diagnosi, con classificazione, stadiazione e storico dei pazienti affetti da melanoma TNM e cSCC TNM Testa-Collo.

Capitolo 2

Analisi dei Requisiti

Una delle prime attività fondamentali dell'ingegneria del software per la buon riuscita di un prodotto, è l'analisi dei requisiti. Questa fase ha lo scopo di individuare e definire le funzionalità offerte che saranno sviluppate.

La nostra applicazione mobile dovrà quindi offrire:

- Fase di apertura, con caricamento dell'interfaccia principale per l'inserimento dei parametri
- Tema dinamico, tramite pulsante per il passaggio da tema chiaro a scuro e viceversa, anche in base alle impostazioni del dispositivo
- Multilingua, inglese e italiano, anch'esse in base alle impostazioni del dispositivo
- Risultati con spiegazione delle analisi, con eventuale follow-up consigliato
- File di configurazione con cui definire parametricamente i risultati che gli input dovranno dare
- Funzionalità di salvataggio dei risultati di diagnosi dei pazienti, con possibilità di visualizzarli in uno storico con ogni loro diagnosi
- Funzionalità di cancellazione dei paziente, inclusi il loro storico, o delle loro singole diagnosi
- Funzionalità per passare da una diagnosi TNM a sSCC Testa-Collo e viceversa
- Funzionalità di caricamento delle diagnosi precedenti, senza dover reinserire i parametri manualmente

- Sincronizzazione con un servizio cloud, per salvare i dati dei pazienti e le loro diagnosi

L'app mobile deve inoltre soddisfare dei requisiti non funzionali, come:

- UI/UX semplice, intuitiva e moderna, senza complicare più del necessario l'utilizzo dell'utente finale
- Sicurezza, sia locale che cloud
- Funzionalità offline, in assenza di internet l'app deve comunque permettere di eseguire diagnosi

Capitolo 3

Progettazione

Successivamente alla fase di analisi dei requisiti, la fase di progettazione definisce come tali requisiti saranno effettivamente implementati, sia a livello di struttura software che di interfaccia utente.

3.1 Descrizione generale

3.2 Scelte architetturali

3.2.1 Migrazione verso Kotlin Multiplatform

Il progetto è stato inizialmente sviluppato in Kotlin nativo Android, utilizzando strumenti Java e non solo, come:

- EvalEx, per la lettura dei file delle espressioni per ottenere le diagnosi
- Firebase, per la sincronizzazione dei dati dei pazienti e delle diagnosi
- SpashScreen, per l'animazione iniziale di caricamento
- Lingua e tema di default in base al dispositivo

Questi elementi sono stati sostituiti con altre librerie simili o sostituiti, nello specifico:

- EvalEx è stato rimosso, e si è invece scritto un interprete da zero nativo Kotlin
- Firebase è stato sostituito con la version Gitlive della community, con l'aggiunta di diverse dipendenze per i servizi Google

- Lo SplashScreen animato è stato rimosso in favore di uno statico
- La lingua e il tema di default sono stati scritti nelle rispettive iosMain e androidMain, in quanto diverse per piattaforma

3.2.2 Architettura dell'applicazione

L'architettura del software si adatta agli standard di Kotlin Multiplatform (KMP). Durante la migrazione da Kotlin a Kotlin Multiplatform, l'obiettivo primario è stato quello di massimizzare il codice condiviso tra Android e iOS, con il minor numero di dipendenze native ove possibile, con una logica di business e di presentazione (UI) unificati.

L'applicazione segue una variante del pattern architetturale **Model-View-ViewModel (MVVM)**, costruita attorno a una singola *ViewModel* centralizzata (`MelanomaViewModel`). Questa funge da singolo punto di accesso per l'intero ciclo di vita dell'applicazione, separando l'interfaccia utente dalla logica di dominio.

Le caratteristiche architetturali principali includono:

- Utilizzo di Compose Multiplaform, compatibile sia con Android che iOS in Kotlin Multiplatform, tramite il modulo condiviso `commonMain`. Le viste (View) sono *stateless*, e si limitano quindi a osservare i dati e a inoltrare al ViewModel le interazioni dell'utente.
- Utilizzo di `StateFlow` e di `MutableStateFlow` per la gestione della reattività degli stati. Variabili come dati anagrafici, parametri clinici e risultati della stadiazione (`StagingResult`) vengono aggiornati reattivamente. Viene quindi ricomposta automaticamente l'interfaccia grafica quando i flussi di dati cambiano, emettendo un nuovo stato.
- Navigazione tra le schermate gestita tramite stato (State-Driven), senza librerie di routing esterne (inizialmente nella versione nativa veniva utilizzato il Navigation Component di Android), viene quindi gestita in modo manuale e dichiarativo la navigazione tra le schermate. Una `Sealed class Screen` definisce le possibili destinazioni navigabili dell'app (`Calculator`, `PatientList` e `PatientHistory`). Il ViewModel mantiene lo stato della schermata attiva, e fornisce la funzione `navigateTo(screen: Screen)`.

Cambiando lo stato della variabile, viene quindi cambiata la schermata, eseguendo eventuale logica annessa (come il caricamento dei pazienti tramite fetch).

- Utilizzo di `Kotlin Coroutines` per la gestione della concorrenza e dell'accesso ai dati. Tutte le operazioni asincrone, come il calcolo della diagnosi basato su regole dei file JSON (e tramite `StagingEngine`), inclusa la comunicazione cloud di Firebase, sono gestite in modo sicuro e non bloccante, anche tramite il `viewModelScope`. L'interazione con il database cloud Firebase, inizialmente con Firebase nativo di Google, è stata migrata alla versione KMP della community GitLive, che permette di utilizzare funzioni CRUD (lettura, salvataggio e cancellazione) sui dati senza dover applicare codici diversi in base al sistema operativo.

In breve, la maggior parte del codice si trova in commonMain e utilizza codice Kotlin compatibile sia con sistemi Android che iOS, con il minimo utilizzo di funzioni proprietare tramite `expect/actual`, lasciando quindi i moduli `androidMain` e `iosMain` il più snelli possibili (salvo inizializzazioni e configurazioni iniziali obbligatorie).

3.3 Specifica dei requisiti

3.3.1 Interfaccia utente

3.3.2 Realizzazione del layout

3.4 Database

3.4.1 Modello dati

Capitolo 4

Sviluppo

4.1 Creazione base dei dati

4.2 Sicurezza

Capitolo 5

Conclusioni